UD1 / REPASO FINAL

Repaso final Python

Clases · CSV · Funciones
Menús · Validación · Persistencia

4 proyectos completos · Dificultad superior a R11

Python 3 / Aplicaciones de consola

ANTES DE EMPEZAR

Reglas comunes para los cuatro proyectos

La referencia directa es la **Relación 11**. Estos ejercicios mantienen el formato de aplicaciones de consola, pero requieren más coordinación entre clases, módulos, ficheros y reglas de negocio. Son independientes: no copies un proyecto para cambiarle solo los nombres.

Condiciones de programación

- Utiliza **Python 3** con if __name__ == "__main__": main() como punto de entrada.
- **Ni `print()` ni `input()` dentro de clases, métodos, ni funciones auxiliares.** Toda la interacción por consola debe permanecer en main(). Los métodos y funciones devolverán datos, valores booleanos, listas, cadenas o códigos de estado.
- Cuando uses un menú, repítelo con una **condición explícita**, sin while True combinado con break para terminar.
- Separa cada ejercicio en al menos main.py y modelos.py; crea persistencia.py cuando te ayude a organizar la lectura/escritura.
- Usa try/except ValueError para la conversión de entradas; no permitas cantidades negativas, claves repetidas ni operaciones que dejen los datos incoherentes.
- Trabaja con csv, with open(...), encoding="utf-8" y newline="". Puedes utilizar pathlib y las colecciones del lenguaje, **sin bases de datos, pandas, APIs ni módulos de Odoo**.

Comprobación del funcionamiento

Cada ejercicio incluye casos para **probar el programa**. Ejecútalos y verifica los resultados, tanto en las operaciones válidas como en las inválidas y tras volver a cargar los CSV.

Nivel esperado

Cada proyecto requiere combinar **POO, listas o diccionarios, funciones, bucles, ficheros CSV y validación**. Se valorará que el diseño impida estados imposibles, no que acumule clases innecesarias.

PYTHON / PROYECTO 01

Floristería Lila: inventario y ventas

Una tienda registra flores y complementos y prepara ventas con varias líneas. A diferencia de un inventario simple, **añadir líneas a una venta afecta al stock** y la venta completa debe calcular su importe.

Modelo de datos

Define Producto(codigo, nombre, categoria, precio, stock), LineaVenta(producto, cantidad, precio_unitario) y Venta(numero, cliente); utiliza una clase gestora si te ayuda. Una venta contiene una lista de líneas; cada línea conserva el precio en el momento de la operación.

Funcionalidad obligatoria

- **1.** Carga un catálogo inicial desde productos.csv, o usa datos ficticios iniciales si no existe: FLO01 Rosas (3,50 €, 15 unidades), FLO02 Tulipanes (2,25 €, 24), FLO03 Orquídeas (18 €, 4).
- **2.** Permite alta de producto y reposición. Rechaza códigos repetidos, precios negativos y stocks negativos.
- **3.** Abre una venta para un cliente ficticio y añade varias líneas. **No modifiques el stock** cuando una cantidad sea inválida o supere lo disponible; ante una línea válida, descuéntalo una sola vez.
- **4.** Muestra el detalle, el subtotal y el total de una venta. Aplica un **10 % de descuento al total** cuando el subtotal sea igual o superior a **50 €**; no apliques descuento por cada línea.
- **5.** Al cerrar una venta, añádela al historial. Permite listar ventas cerradas y el importe acumulado de todas ellas.
- **6.** Guarda el catálogo y las ventas en CSV; al ejecutar nuevamente el programa, recupera el stock actualizado y el historial de ventas.

Menú mínimo

1) Ver productos · 2) Alta · 3) Reponer · 4) Nueva venta / añadir líneas · 5) Cerrar venta · 6) Consultar ventas · 7) Guardar · 0) Salir.

Persistencia y estructura

main.py, modelos.py, persistencia.py, productos.csv y ventas.csv. Decide cómo representar las líneas en el CSV para poder reconstruir las ventas al cargar (por ejemplo, una fila por línea).

Pruebas que debe superar

- Con 15 rosas, vender 3 reduce el stock a 12; pedir 20 no lo modifica.
- Una venta de **2 orquídeas + 4 tulipanes** tiene subtotal **45 €**, sin descuento.
- En otra prueba con stock inicial, una venta de **3 orquídeas** da subtotal **54 €** y total **48,60 €**.
- Tras guardar y reiniciar, el catálogo y las ventas cerradas siguen siendo coherentes.

Ampliación voluntaria

Añade una opción de cancelar la venta actual **devolviendo al stock** todas las unidades reservadas, sin alterar ventas ya cerradas.

PYTHON / PROYECTO 02

Colección de cartas Pokémon: intercambios

Una persona organiza su colección de cartas y registra intercambios. El objetivo es practicar búsqueda, validación de duplicados, cálculos agregados y una operación que debe ser **atómica**: o se realiza por completo o no cambia nada.

Modelo de datos

Define Carta(codigo, nombre, tipo, rareza, valor) y Coleccion con una colección de cartas. El código identifica de forma única cada carta; también puedes registrar una Coleccion mediante un diccionario por código.

Funcionalidad obligatoria

- **1.** Carga la colección desde cartas.csv o inicia con PK01 Pikachu (Eléctrico, Común, 5 €), PK02 Charizard (Fuego, Rara, 30 €), PK03 Bulbasaur (Planta, Común, 4 €).
- **2.** Permite añadir una carta y eliminarla por su código. No admitas códigos vacíos o repetidos ni valores negativos.
- **3.** Implementa búsqueda por nombre o tipo (sin distinguir mayúsculas/minúsculas) y filtrado por rareza.
- **4.** Permite **intercambiar** una carta existente por una carta nueva: primero comprueba que la antigua existe y que el código nuevo no está ocupado; solo entonces elimina una e incorpora la otra.
- **5.** Calcula y muestra el **valor total** de la colección, el número de cartas y cuántas hay de cada rareza.
- **6.** Guarda y recupera la colección desde CSV. Comprueba que se conservan las tildes.

Menú mínimo

1) Listar · 2) Añadir · 3) Eliminar · 4) Buscar/filtrar · 5) Intercambiar · 6) Estadísticas · 7) Guardar · 0) Salir.

Persistencia y estructura

main.py, modelos.py, persistencia.py, cartas.csv; CSV con cabecera codigo,nombre,tipo,rareza,valor (admite el separador correctamente configurado).

Pruebas que debe superar

- Con las tres cartas iniciales el valor es **39 €**.
- Un alta con código PK01 repetido no modifica la colección.
- Intercambiar PK02 por otra carta con código PK03 debe fallar **sin perder PK02**.
- Un intercambio válido reemplaza exactamente una carta y recalcula el valor total.
- Guardar, reiniciar y cargar recupera la misma lista sin duplicados.

Ampliación voluntaria

Ordena las cartas por valor descendente y permite consultar las tres de mayor valor.

PYTHON / PROYECTO 03

Centro de talleres: inscripciones y plazas

Una academia organiza talleres con aforo limitado. Se debe evitar que un participante se inscriba dos veces y que un taller acepte más personas de las permitidas.

Modelo de datos

Crea Participante(email, nombre), Taller(codigo, titulo, plazas) e Inscripcion(email_participante, codigo_taller). Puedes gestionar las inscripciones desde una clase CentroTalleres, relacionándolas con participantes y talleres existentes.

Funcionalidad obligatoria

- **1.** Prepara al menos tres talleres con códigos distintos (por ejemplo: Cerámica, Fotografía y Cocina), con aforos **2, 3 y 4**. Permite dar de alta participantes por correo electrónico.
- **2.** Permite inscribir a una persona existente en un taller existente. Rechaza emails repetidos en el alta, inscripciones duplicadas y talleres que ya estén completos.
- **3.** Permite cancelar una inscripción. Cancelar una inscripción inexistente no debe afectar a ninguna otra.
- **4.** Consulta los inscritos por taller y calcula las plazas libres y el porcentaje de ocupación. Si un taller tiene 2 plazas y 1 inscrito, mostrará **1 plaza libre y 50 %** de ocupación.
- **5.** Permite buscar un participante por fragmento del nombre y consultar los talleres a los que está inscrito.
- **6.** Guarda participantes, talleres e inscripciones en **tres CSV** y recupéralos al inicio; no permitas que una carga genere inscripciones a emails o talleres inexistentes.

Menú mínimo

1) Participantes · 2) Alta participante · 3) Inscribir · 4) Cancelar · 5) Ver taller · 6) Buscar participante · 7) Guardar · 0) Salir.

Persistencia y estructura

main.py, modelos.py, persistencia.py, participantes.csv, talleres.csv e inscripciones.csv. Relaciona registros mediante **códigos y emails**, no copiando un objeto completo en el CSV.

Pruebas que debe superar

- La tercera inscripción en el taller de 2 plazas es rechazada.
- Inscribir dos veces el mismo email en un taller solo deja **una inscripción**.
- Cancelar una inscripción válida libera una plaza.
- Un email/taller inexistente no genera ninguna inscripción.
- Tras reiniciar, se mantienen talleres, plazas y participantes correctamente vinculados.

Ampliación voluntaria

Muestra un ranking de talleres ordenado por porcentaje de ocupación.

PYTHON / PROYECTO 04

Servicio técnico: órdenes y herencia

Gestiona reparaciones y asistencia remota para clientes ficticios. Dos tipos de trabajo comparten datos y estados, pero calculan sus importes de manera distinta. Este proyecto recupera expresamente la **herencia de la R10**.

Modelo de datos

Define una clase base Trabajo(codigo, cliente, horas, estado) y dos hijas: Reparacion, que además tiene precio_pieza, y AsistenciaRemota. En ambas debe existir calcular_importe(), con su cálculo correspondiente. Mantén una lista o diccionario de trabajos.

Funcionalidad obligatoria

- **1.** Da de alta trabajos con un código único, horas mayores que cero y un cliente no vacío. Rechaza tipos desconocidos y precios de pieza negativos.
- **2.** Calcula el importe de una **Reparación** como horas × 25 € + precio_pieza, y el de **Asistencia remota** como horas × 20 €.
- **3.** Gestiona los estados **pendiente → en curso → finalizado**. No permitas saltar directamente de pendiente a finalizado ni retroceder de estado.
- **4.** Permite listar trabajos y buscar por cliente, filtrando opcionalmente por tipo o estado. Muestra también el importe individual de cada trabajo.
- **5.** Presenta un resumen con el **número de trabajos por estado** y el **importe total** de los trabajos finalizados.
- **6.** Guarda y recarga desde un CSV con un campo tipo para reconstruir el subtipo correcto. Conservar el tipo tras cargar es parte del ejercicio.

Menú mínimo

1) Listar · 2) Alta reparación · 3) Alta remota · 4) Cambiar estado · 5) Buscar/filtrar · 6) Resumen · 7) Guardar · 0) Salir.

Persistencia y estructura

main.py, modelos.py, persistencia.py, trabajos.csv. Puedes dejar precio_pieza vacío o a 0 en los trabajos remotos, siempre que el formato pueda volver a cargarse.

Pruebas que debe superar

- Reparación de **2 horas** y pieza de **40 €**: importe **90 €**.
- Asistencia remota de **1,5 horas**: importe **30 €**.
- Un código repetido no crea otro trabajo.
- La transición pendiente → finalizado se rechaza sin modificar el estado.
- Tras reiniciar, se conservan correctamente los subtipos y sus importes.

Ampliación voluntaria

Permite cambiar las horas de un trabajo no finalizado y recalcula su importe; impide editar trabajos finalizados.

AUTOEVALUACIÓN / PYTHON

Revisión final

- ¿Todos los menús terminan con la opción 0 y una condición explícita?
- ¿Las funciones y los métodos **devuelven resultados** en vez de imprimir o pedir datos?
- ¿Los errores de entrada se resuelven sin cerrar inesperadamente el programa?
- ¿Los códigos y emails se mantienen únicos según cada supuesto?
- ¿Las operaciones fallidas no dejan cambios parciales en stock, colección, plazas o estado?
- ¿La lectura CSV convierte correctamente números y estados a los tipos necesarios?
- ¿Después de guardar, cerrar y volver a ejecutar puedes recuperar los datos sin duplicarlos?

Puedes utilizar el ZIP de archivos base para empezar con la estructura de carpetas. Los archivos **no contienen soluciones**.

Relaciones de referencia: R5-R10 para técnicas aisladas y R11 para proyectos integradores. La complejidad adicional reside en combinar reglas y persistencia, no en utilizar bibliotecas nuevas.